

Deterministic Optimization

Discrete Optimization:
Modeling w/ binary
variables

Shabbir Ahmed

Anderson-Interface Chair and Professor
School of Industrial and Systems Engineering

Set Packing, Covering,
Partitioning

Set Packing, Covering and Partitioning

Learning Objectives

- Discover integer programming formulations of set based models
- Observe a python example of modeling and solving such a model

A Store Location Example

- Budget Stores is considering to locate superstores in 4 cities
- Its customer base is spread across 6 neighboring cities
- Customers from a city that is within driving distance from a store will visit that store
- Budget does not want to locate two or more stores that are with driving distance of the same customer city (to avoid cannibalization of the customer base)
- Budget wants to maximize the number of customers it can attract

Example: Data

City	Location A	Location B	Location C	Location D	Customers
A	1	1	0	1	100
B	1	1	1	0	50
C	0	1	1	0	70
D	1	0	0	1	150
E	1	1	1	0	20
F	0	1	0	1	80

Cell (i, j) has an entry of 1 if city i is within driving distance of location j .

Example: Data

City	Location A	Location B	Location C	Location D	Customers
A	1	1	0	1	100
B	1	1	1	0	50
C	0	1	1	0	70
D	1	0	0	1	150
E	1	1	1	0	20
F	0	1	0	1	80
Total	320	320	140	330	

Model and Code

$$\begin{array}{ll}\max & 320x_A + 320x_B + 140x_C + 330x_D \\ \text{s.t.} & x_A + x_B + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_C \leq 1 \\ & x_A + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_D \leq 1 \\ & x_A, x_B, x_C, x_D \in \{0, 1\}\end{array}$$

Model and Code

$$\begin{aligned} \max \quad & 320x_A + 320x_B + 140x_C + 330x_D \\ \text{s.t.} \quad & x_A + x_B + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_C \leq 1 \\ & x_A + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_D \leq 1 \\ & x_A, x_B, x_C, x_D \in \{0, 1\} \end{aligned}$$

```
1 # Store Location using a
2 # Set Packing Formulation
3 # Install PuLP and glpk before running this code
4 # http://pythonhosted.org/PuLP/
5 # https://www.gnu.org/software/glpk/
6
7 import pulp as p
8
9 # create a model
10 m = p.LpProblem("Store Location",p.LpMaximize)
11
12 # Create decision variables
13 xA = p.LpVariable('xA',0,1,p.LpInteger)
14 xB = p.LpVariable('xB',0,1,p.LpInteger)
15 xC = p.LpVariable('xC',0,1,p.LpInteger)
16 xD = p.LpVariable('xD',0,1,p.LpInteger)
17
18 # Add objective and constraints
19 m += 320*xA + 320*xB + 140*xC + 330*xD, "Objective"
20 m += xA + xB + xD <= 1, "Customer A"
21 m += xA + xB + xC <= 1, "Customer B"
22 m += xB + xC <= 1, "Customer C"
23 m += xA + xD <= 1, "Customer D"
24 m += xA + xB + xC <= 1, "Customer E"
25 m += xB + xD <= 1, "Customer F"
26
27 m.solve()
28 for v in m.variables():
29     print(v.name, "=", v.varValue)
```

Model and Code

$$\begin{aligned} \max \quad & 320x_A + 320x_B + 140x_C + 330x_D \\ \text{s.t.} \quad & x_A + x_B + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_C \leq 1 \\ & x_A + x_D \leq 1 \\ & x_A + x_B + x_C \leq 1 \\ & x_B + x_D \leq 1 \\ & x_A, x_B, x_C, x_D \in \{0, 1\} \end{aligned}$$

```
('xA', '=', 0.0)
('xB', '=', 0.0)
('xC', '=', 1.0)
('xD', '=', 1.0)
```

```
1 # Store Location using a
2 # Set Packing Formulation
3 # Install PuLP and glpk before running this code
4 # http://pythonhosted.org/PuLP/
5 # https://www.gnu.org/software/glpk/
6
7 import pulp as p
8
9 # create a model
10 m = p.LpProblem("Store Location",p.LpMaximize)
11
12 # Create decision variables
13 xA = p.LpVariable('xA',0,1,p.LpInteger)
14 xB = p.LpVariable('xB',0,1,p.LpInteger)
15 xC = p.LpVariable('xC',0,1,p.LpInteger)
16 xD = p.LpVariable('xD',0,1,p.LpInteger)
17
18 # Add objective and constraints
19 m += 320*xA + 320*xB + 140*xC + 330*xD, "Objective"
20 m += xA + xB + xD <= 1, "Customer A"
21 m += xA + xB + xC <= 1, "Customer B"
22 m += xB + xC <= 1, "Customer C"
23 m += xA + xD <= 1, "Customer D"
24 m += xA + xB + xC <= 1, "Customer E"
25 m += xB + xD <= 1, "Customer F"
26
27 m.solve()
28 for v in m.variables():
29     print(v.name, "=", v.varValue)
```

Set Packing: General Formulation

Given m elements and a collection of subsets $S_1, \dots, S_n \subseteq \{1, \dots, m\}$ with associated nonnegative weights $w_1, \dots, w_n$, pick sets from this collection such that they are disjoint and the sum of the weights is maximized.

$$\begin{aligned} \max \quad & \sum_{j=1}^n w_j x_j \\ \text{s.t.} \quad & \sum_{j: i \in S_j} x_j \leq 1 \quad \forall i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad \forall j = 1, \dots, n. \end{aligned}$$

Set Covering: General Formulation

Given m elements and a collection of subsets $S_1, \dots, S_n \subseteq \{1, \dots, m\}$ with associated nonnegative weights $w_1, \dots, w_n$, pick sets from this collection such that they cover each item and the sum of the weights is minimized.

$$\begin{aligned} \min \quad & \sum_{j=1}^n w_j x_j \\ \text{s.t.} \quad & \sum_{j: i \in S_j} x_j \geq 1 \quad \forall i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad \forall j = 1, \dots, n. \end{aligned}$$

Set Partitioning: General Formulation

Given m elements and a collection of subsets $S_1, \dots, S_n \subseteq \{1, \dots, m\}$ with associated nonnegative weights $w_1, \dots, w_n$, pick sets from this collection such that they cover exactly one item and the sum of the weights is maximized/minimized.

$$\begin{array}{ll} \max / \min & \sum_{j=1}^n w_j x_j \\ \text{s.t.} & \sum_{j: i \in S_j} x_j = 1 \quad \forall i = 1, \dots, m \\ & x_j \in \{0, 1\} \quad \forall j = 1, \dots, n. \end{array}$$

Summary

- Set packing/covering/partitioning models arise in many applications where decisions involve choosing subsets of a given set of items
- We also observed an example of using PuLP and glpk to solve an integer programming formulation of such a problem